Introduction to PyTorch

In this notebook you will learn the fundamentals of PyTorch — the most widely used deep learning framework in research and industry.

By the end of this notebook you will be able to:

- Create and manipulate **tensors** (PyTorch's core data structure)
- Reshape tensors confidently
- Build a **custom Dataset** and load data with a **DataLoader**
- Apply **data augmentation** and preprocessing transforms

📌 **Prerequisites:** basic Python and NumPy familiarity.

1. What is PyTorch?

PyTorch is an open-source deep learning framework developed by Meta AI. It is built around two key ideas:

1. **Tensor computation** — like NumPy, but with GPU acceleration.
2. **Automatic differentiation** — PyTorch tracks operations on tensors so it can compute gradients automatically (you will use this heavily when training models in the next notebook).

To install PyTorch, visit https://pytorch.org/get-started/locally/ and follow the instructions for your system.

```
# Typical CPU-only install
pip install torch torchvision
```

2. Working with Tensors

A **tensor** is the fundamental data structure in PyTorch — think of it as a generalisation of a **N-dimensional** array:

Concept	NumPy	PyTorch
1-D array	`np.array([1,2,3])`	`torch.tensor([1,2,3])`
2-D array	`np.zeros((3,4))`	`torch.zeros((3,4))`
N-D array	`ndarray`	`Tensor`

The key advantage over NumPy: tensors can live on a **GPU** and support automatic differentiation.

2.1 Creating Tensors

Fill in the missing function names to create each type of tensor:

```
import torch

# Random values sampled from a uniform distribution [0, 1)
tensor_random = torch.rand((2, 3))

# Tensor filled with zeros
tensor_zeros = torch.zeros((4, 4))

# Tensor filled with ones
tensor_ones = torch.ones((3, 3))

# Tensor created directly from a Python list
tensor_list = torch.tensor([[1, 2, 3], [4, 5, 6]])

print("Random:\n", tensor_random)
print("Zeros:\n", tensor_zeros)
print("Ones:\n", tensor_ones)
print("From list:\n", tensor_list)
```

```
Random:
 tensor([[0.2700, 0.7778, 0.1777],
        [0.2177, 0.8226, 0.5598]])
Zeros:
 tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
Ones:
 tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
From list:
 tensor([[1, 2, 3],
        [4, 5, 6]])
```

2.2 Basic Tensor Operations

Fill in the correct operators and function name:

- Element-wise addition: `c = a _____ b`
- Element-wise multiplication: `d = a _____ b`
- Matrix multiplication: `e = torch._____(a, b)`

```
a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
b = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

# Element-wise addition — each element added independently
```

```
c = a + b

# Element-wise multiplication (NOT matrix multiplication)
d = a * b

# Matrix multiplication — rows × columns, standard linear-algebra sense
e = torch.matmul(a, b)

print("a + b:\n", c)
print("a * b (element-wise):\n", d)
print("a @ b (matmul):\n", e)
```

```
a + b:
 tensor([[ 6.,  8.],
        [10., 12.]])
a * b (element-wise):
 tensor([[ 5., 12.],
        [21., 32.]])
a @ b (matmul):
 tensor([[19., 22.],
        [43., 50.]])
```

2.3 Inspecting Tensor Properties

Three attributes you will check constantly when debugging:

- `.shape` — dimensions of the tensor (e.g. `torch.Size([2, 3])`)
- `.dtype` — element data type (e.g. `torch.float32`)
- `.device` — where the tensor lives (`cpu` or `cuda:0`)

Fill in the attribute names:

```
tensor = torch.rand(2, 3)

print("Shape:", tensor.shape)
print("Data Type:", tensor.dtype)
print("Device:", tensor.device)
```

```
Shape: torch.Size([2, 3])
Data Type: torch.float32
Device: cpu
```

2.4 Moving Tensors Between CPU and GPU

GPU training can be 10–100× faster than CPU for large models. PyTorch makes it easy to move data between devices. The standard pattern is to detect the available device once at the top of your script and then move everything there.

Fill in the missing method name and variable:

```
# Detect device — falls back to CPU gracefully if no GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

tensor_gpu = tensor.to(device)
print("Tensor device:", tensor_gpu.device)
```

```
Using device: cuda
Tensor device: cuda:0
```

3. Tensor Reshaping

Reshaping is one of the most common operations you will perform — for example, flattening an image before passing it to a linear layer, or adding a batch dimension.

Method	What it does
`.view(shape)`	Reinterpret the data with a new shape — **zero-copy**, requires contiguous memory
`.reshape(shape)`	Same as view but works even if memory is non-contiguous (safer default)
`.squeeze()`	Remove all dimensions of size 1
`.unsqueeze(dim)`	Insert a new dimension of size 1 at position `dim`
`.permute(dims)`	Reorder dimensions (useful when converting between channel-first and channel-last)

💡 Use `-1` in a shape to let PyTorch infer that dimension automatically.

Fill in the blanks to perform the described reshape operations:

```
t = torch.arange(24)   # 1-D tensor: [0, 1, 2, ..., 23]
print("Original shape:", t.shape)   # torch.Size([24])

# Reshape to 2-D: 4 rows, 6 columns
t_2d = t.reshape(4, 6)
print("Reshaped to (4, 6):\n", t_2d)

# Flatten back to 1-D — use -1 to infer the size automatically
t_flat = t_2d.reshape(-1)
print("Flattened:", t_flat.shape)   # torch.Size([24])

# Reshape to 3-D: batch=2, height=3, width=4
t_3d = t.reshape(2, 3, 4)
print("Reshaped to (2, 3, 4):\n", t_3d)
```

```
Original shape: torch.Size([24])
Reshaped to (4, 6):
 tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23]])
Flattened: torch.Size([24])
Reshaped to (2, 3, 4):
```

```
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]])
```

Fill in the blanks for `squeeze` and `unsqueeze`:

```
t = torch.rand(3, 1, 4)   # middle dimension is 1
print("Original shape:", t.shape)          # (3, 1, 4)

# Remove all dimensions of size 1
t_sq = t.squeeze()
print("After squeeze:", t_sq.shape)        # (3, 4)

# Add a batch dimension of size 1 at position 0
t_unsq = t_sq.unsqueeze(0)
print("After unsqueeze(0):", t_unsq.shape) # (1, 3, 4)
```

```
Original shape: torch.Size([3, 1, 4])
After squeeze: torch.Size([3, 4])
After unsqueeze(0): torch.Size([1, 3, 4])
```

Fill in the `permute` call to convert from channel-first `(B, C, H, W)` to channel-last `(B, H, W, C)`:

```
# PyTorch uses channel-first: (batch, channels, height, width)
# Some libraries use channel-last: (batch, height, width, channels)

img = torch.rand(8, 3, 32, 32)        # (B, C, H, W)
img_hwc = img.permute(0, 2, 3, 1)     # → (B, H, W, C)
print("Channel-first:", img.shape)
print("Channel-last: ", img_hwc.shape)
```

```
Channel-first: torch.Size([8, 3, 32, 32])
Channel-last:  torch.Size([8, 32, 32, 3])
```

4. Datasets and DataLoaders

PyTorch separates **what your data is** from **how it is served to the model**:

- A **`Dataset`** defines *what* your data looks like — it knows how many samples there are and how to retrieve the i-th one.
- A **`DataLoader`** wraps a Dataset and handles *how* data is delivered — batching, shuffling, parallel loading.

```
Dataset  →  DataLoader  →  Training loop
 (storage)    (batching)
```

A Dataset represents the **underlying** data, defining how each sample is stored and accessed.

A DataLoader helps in **batching** the data into manageable batches for training.

Why not just load everything into a list?

Real datasets (ImageNet: 1.2 M images) do not fit in RAM. The Dataset + DataLoader pattern lets you load only what you need, when you need it.

5. Creating a Custom Dataset

To create a custom Dataset, subclass `torch.utils.data.Dataset` and implement exactly **two methods**:

- `__len__()` — returns the total number of samples
- `__getitem__(index)` — returns the sample at position `index`

The DataLoader will call these methods internally. Fill in the missing return value:

```
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os

class CustomImageDataset(Dataset):
    def __init__(self, image_dir, transform=None):
        """
        Args:
            image_dir: path to the folder containing images
            transform: optional torchvision transform applied to each image
        """
        self.image_dir = image_dir
        self.image_files = os.listdir(image_dir)
        self.transform = transform

    def __len__(self):
        # The DataLoader needs to know how many samples exist
        return len(self.image_files)

    def __getitem__(self, index):
        img_path = os.path.join(self.image_dir, self.image_files[index])
        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image
```

6. Using DataLoader

Key `DataLoader` parameters:

Parameter	Description
`batch_size`	How many samples per batch
`shuffle`	Randomise order each epoch (use `True` for training, `False` for validation)
`num_workers`	Number of parallel processes for loading (0 = main process only)

Fill in sensible values for a training DataLoader:

```
# dataset = CustomImageDataset(image_dir="path/to/images")

# dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=2)

# # Iterate over batches
# for batch in dataloader:
#     print(batch.shape)  # Expected: (batch_size, 3, H, W) after transforms
#     break

print("Uncomment the code above once you have a real image directory.")
```

```
Uncomment the code above once you have a real image directory.
```

7. Loading a Built-in Dataset (MNIST)

`torchvision.datasets` ships with many standard datasets (MNIST, CIFAR-10, ImageNet, ...). This is the quickest way to get started without managing files yourself.

Before loading, we define a **transform pipeline** — a sequence of preprocessing steps applied to every sample on the fly.

Fill in the correct transform names and the expected dataset length:

```
import torchvision.transforms as transforms
from torchvision import datasets

transform = transforms.Compose([
    transforms.ToTensor(),              # Converts PIL Image [0,255] → float tensor [0,1
    transforms.Normalize(0.5, 0.5)      # Normalises range from [0,1] → [-1,1]
])

train_dataset = datasets.MNIST(root='data', train=True, transform=transform, download=Tru

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

print("Training samples:", len(train_dataset))  # Expected: 60000
print("Number of batches:", len(train_loader))
```

```
# Inspect a single batch
images, labels = next(iter(train_loader))
print("Batch shape:", images.shape)   # Expected: (32, 1, 28, 28)
print("Labels shape:", labels.shape)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 17.6MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 481kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.43MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.58MB/s]Training samples: 60000
Number of batches: 1875
Batch shape: torch.Size([32, 1, 28, 28])
Labels shape: torch.Size([32])
```

8. Data Augmentation & Preprocessing

Data augmentation artificially increases the diversity of your training set by applying random transformations — without collecting new data. Common benefits:

- Reduces overfitting
- Makes the model more robust to real-world variation (different lighting, orientations, etc.)

⚠️ Important: augmentations should only be applied to the **training set**, not to validation/test sets.

Fill in the missing transform names:

```
import torchvision.transforms as transforms

train_transform = transforms.Compose([
    transforms.Resize((128, 128)),                              # Resize all images to th
    transforms.RandomHorizontalFlip(),                          # 50% chance of flipping
    transforms.RandomRotation(degrees=15),                      # Random rotation ±15°
    transforms.ColorJitter(brightness=0.2, contrast=0.2),      # Random brightness/contra
    transforms.ToTensor(),                                      # PIL → float tensor [0,
    transforms.Normalize(mean=[0.5], std=[0.5])                 # Normalise to [-1, 1]
])

# Validation transform — no random augmentation, just resize and normalise
val_transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])
```

Applying Transforms and Exploring the Dataset

Let's make this concrete. Below we apply three different transform pipelines to MNIST and compare what comes out:

1. **No transform** — raw PIL image
2. **Minimal** — `ToTensor()` only
3. **Full augmentation** — resize, flip, rotation, normalise

We then inspect dataset size, batch shapes, pixel value ranges, and visualise a sample batch.

```
import torch
import torchvision.transforms as transforms
from torchvision import datasets
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

# ── 1. No transform (raw PIL) ──────────────────────────────────────────
raw_dataset = datasets.MNIST(root='data', train=True, transform=None, download=True)

# ── 2. Minimal: ToTensor only ──────────────────────────────────────────
minimal_transform = transforms.Compose([
    transforms.ToTensor()   # PIL [0,255] uint8 → float32 tensor [0,1]
])
minimal_dataset = datasets.MNIST(root='data', train=True, transform=minimal_transform, do

# ── 3. Full augmentation pipeline ──────────────────────────────────────
augment_transform = transforms.Compose([
    transforms.Resize((32, 32)),             # Upsample from 28×28 to 32×32
    transforms.RandomHorizontalFlip(),       # 50% chance of left-right flip
    transforms.RandomRotation(degrees=15),   # Random rotation ±15°
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])  # [0,1] → [-1,1]
])
augmented_dataset = datasets.MNIST(root='data', train=True, transform=augment_transform,

# Also load the test split for comparison
test_dataset = datasets.MNIST(root='data', train=False, transform=minimal_transform, down

# ── Dataset sizes — fill in the blanks ─────────────────────────────────
print("=== Dataset sizes ===")
print(f"Training samples : {len(minimal_dataset):,}")
print(f"Test samples     : {len(test_dataset):,}")
print(f"Total            : {len(minimal_dataset) + len(test_dataset):,}")
print(f"Classes          : {minimal_dataset.classes}")
print(f"Number of classes: {len(minimal_dataset.classes)}")
```

```
=== Dataset sizes ===
Training samples : 60,000
Test samples     : 10,000
Total            : 70,000
Classes          : ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five',
Number of classes: 10
```

```
# ── Inspect individual samples ──────────────────────────────────────────
# Fill in the index to retrieve the first sample from each dataset
```

```
print("=== Single sample inspection ===")

raw_img, label = raw_dataset[0]
print(f"Raw PIL image  : type={type(raw_img).__name__}, size={raw_img.size}, label={labe

min_img, label = minimal_dataset[0]
print(f"Minimal tensor : shape={min_img.shape}, dtype={min_img.dtype}")
print(f"                 min={min_img.min():.3f}, max={min_img.max():.3f}")

aug_img, label = augmented_dataset[0]
print(f"Augmented tensor: shape={aug_img.shape}, dtype={aug_img.dtype}")
print(f"                  min={aug_img.min():.3f}, max={aug_img.max():.3f}")
print()
# Q: Why is the min of the augmented tensor negative but the minimal tensor's min is 0?
# A: Because Normalize(mean=0.5, std=0.5) shifts the pixel range from [0,1] to [-1,1], ma
```

```
=== Single sample inspection ===
Raw PIL image  : type=Image, size=(28, 28), label=5
Minimal tensor : shape=torch.Size([1, 28, 28]), dtype=torch.float32
                 min=0.000, max=1.000
Augmented tensor: shape=torch.Size([1, 32, 32]), dtype=torch.float32
                  min=-1.000, max=0.984
```

```
# ── DataLoader batch shapes ──────────────────────────────────────────────
# Create loaders for both pipelines
loader_minimal   = DataLoader(minimal_dataset,   batch_size=32, shuffle=True)
loader_augmented = DataLoader(augmented_dataset, batch_size=32, shuffle=True)

imgs_min, labels = next(iter(loader_minimal))
print(f"Minimal   batch — images: {imgs_min.shape}, labels: {labels.shape}")
# Expected shape: (32, 1, 28, 28)

imgs_aug, labels = next(iter(loader_augmented))
print(f"Augmented batch — images: {imgs_aug.shape}, labels: {labels.shape}")
# Expected shape: (32, 1, 32, 32)

print(f"\nBatches per epoch (batch_size=32): {len(loader_minimal)}")

# Class distribution in one batch
unique, counts = torch.unique(labels, return_counts=True)
for cls, cnt in zip(unique.tolist(), counts.tolist()):
    print(f"  Class {cls}: {cnt} sample(s)")
```

```
Minimal   batch — images: torch.Size([32, 1, 28, 28]), labels: torch.Size([32])
Augmented batch — images: torch.Size([32, 1, 32, 32]), labels: torch.Size([32])

Batches per epoch (batch_size=32): 1875
  Class 0: 2 sample(s)
  Class 1: 8 sample(s)
  Class 2: 3 sample(s)
  Class 3: 4 sample(s)
  Class 5: 2 sample(s)
  Class 6: 4 sample(s)
```

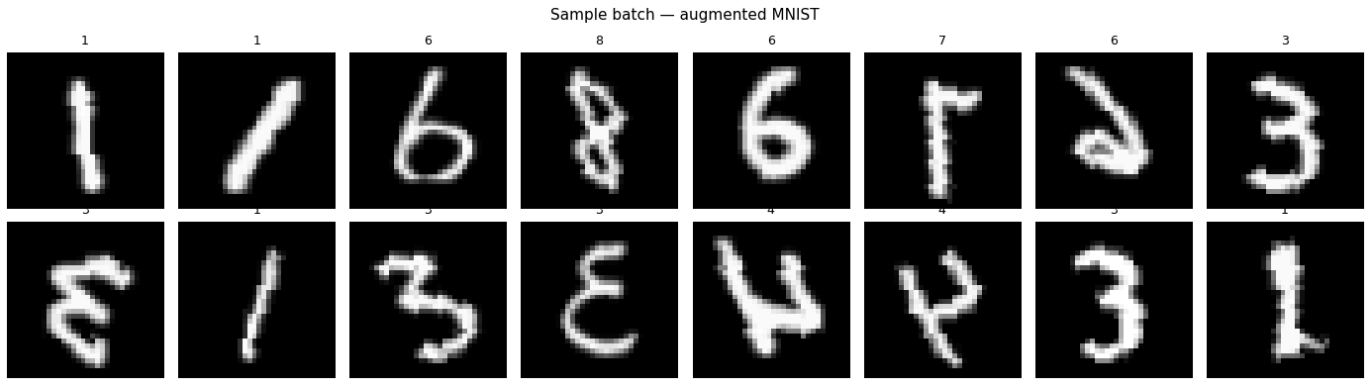
```
  Class 7: 5 sample(s)
  Class 8: 2 sample(s)
  Class 9: 2 sample(s)
```

```
# ── Visualise a batch ──────────────────────────────────────────────────────
imgs, labels = next(iter(loader_augmented))

fig, axes = plt.subplots(2, 8, figsize=(14, 4))
fig.suptitle("Sample batch — augmented MNIST", fontsize=11)

for i, ax in enumerate(axes.flat):
    img = imgs[i].squeeze().numpy()
    # The images are normalised to [-1, 1]. Undo normalisation for display.
    # Formula: original = pixel * std + mean = pixel * 0.5 + 0.5
    img = (img * 0.5) + 0.5    # fill in the correct values
    ax.imshow(img, cmap='gray', vmin=0, vmax=1)
    ax.set_title(str(labels[i].item()), fontsize=9)
    ax.axis('off')

plt.tight_layout()
plt.show()
```



Summary

Concept	Key class / function
Create tensor	`torch.tensor()`, `torch.rand()`, `torch.zeros()`, `torch.ones()`
Reshape tensor	`.reshape()`, `.view()`, `.squeeze()`, `.unsqueeze()`, `.permute()`
Move to GPU	`.to(device)`
Custom dataset	Subclass `Dataset`, implement `__len__` and `__getitem__`

Concept	Key class / function
Load in batches	`DataLoader(dataset, batch_size=…, shuffle=…)`
Preprocessing	`transforms.Compose([…])`

Next notebook: building neural networks with `nn.Module` and writing a training loop.